

Evaluating Hybrid Intelligence in Partially Observable Card Games

No Author Given

¹ Vrije Universiteit Amsterdam

² De Boelelaan 1105, 1081 HV Amsterdam, Holland

³ Intelligent Systems Project 2026

Abstract. This research analyzes the performance of a Hybrid Intelligence (HI) bot created by adding hard-coded, phase-aware heuristics to an existing machine-learning-based Schnapsen bot. Performance was evaluated by comparing win rates of the HI bot to the original ML bot in head-to-head matches and against baseline opponents (RandBot and the search-based RdeepBot). The heuristic layer applies score- and phase-dependent decision rules before falling back to ML predictions. Results show that incorporating heuristics improves win rates across evaluation settings, indicating that combining rule-based decision-making with machine learning can enhance strategic performance in a partially observable, imperfect-information card game. This work contributes empirical evidence for hybrid approaches to game-playing agents in artificial and hybrid intelligence.

Keywords: Artificial Intelligence · Hybrid Intelligence · Machine Learning · Heuristics · Schnapsen · Rule Based · Imperfect Information · Partially Observable · Card Games.

1 Introduction

Hybrid Intelligence, defined as the combination of machine learning (ML) techniques and rule-based heuristics, has attracted significant attention as a means of improving performance in decision-making systems, particularly in complex environments. This study investigates the implementation and evaluation of a hybrid intelligence approach in the Schnapsen card game, a partially observable imperfect-information environment that requires strategic reasoning under uncertainty. By integrating domain-specific heuristics into an existing machine learning based agent, this work aims to assess whether the hybrid approach outperforms the original Machine Learning agent in terms of win rate against two baseline opponents: RandBot, which follows a fully random decision-making strategy, and RdeepBot, a strong search-based agent. Through multiple tournament-based evaluation sets and statistical analysis, this research provides insight into the effectiveness of hybrid intelligence in enhancing strategic gameplay. The results contribute evidence for the applicability of hybrid approaches in similar decision-making contexts and establish a foundation for future work in this area.

Throughout this paper, the terms “agent” and “bot” are used interchangeably to refer to an autonomous decision-making player in Schnapsen.

2 Background Information

2.1 Hybrid Intelligence

Hybrid Intelligence (HI) refers to the combination of machine learning (ML) with rule-based AI systems to take advantage of the strengths of both approaches [10]. Rule-based systems are deterministic and interpretable, making them ideal for tasks requiring well-defined logic and explicit decision making [3]. On the other hand, ML is better at adapting to new data and recognizing complex patterns. This is ideal for scenarios that are rare or where decisions can be ambiguous [4]. By integrating these methods, rule-based HI systems can address scenarios that demand both adaptability and reliability. [1] Recent work on ensemble strategy learning in imperfect-information games further supports this view by showing that combining multiple paradigms (such as learning-based, rule-based, and game-theoretic components) can outperform single-paradigm agents. [10]

Hybrid Intelligence has been found to be effective in other games before. Recent academic work has also explored hybridizing learning-based systems with classical search methods in deterministic games such as Chess and Crazyhouse, demonstrating that combining complementary decision mechanisms can produce measurable performance gains [7]. Beyond such domain-specific systems, contemporary research has shown that a unified algorithm combining guided search, self-play learning, and game-theoretic reasoning can achieve strong performance across both perfect and imperfect information games, further highlighting the general effectiveness of hybrid decision systems [11].

While these approaches primarily focus on hybridization within search-based frameworks, our work investigates hybrid intelligence through the integration of learned policies and explicit domain heuristics in a partially observable card game environment.

In the context of this research, rule-based AI is attractive in situations where certain options are undoubtedly favorable. The bot can be hard-coded to play a certain move if they are in the specified situation. Machine learning is applied when rule-based AI lacks sufficient domain knowledge. In situations that are not specified in the code, ML takes over by analyzing previous games and choosing the option that resulted in favorable outcomes in the past.

2.2 Schnapsen

Schnapsen is a two-player card game that is very popular in Austria and is traditionally played with a 20-card deck. Each player aims to score 66 points by winning tricks, declaring marriages, and making strategic plays. Points are awarded for winning tricks, with higher value given to certain cards and combinations, such as the marriage between a king and queen of the same suit. [5]

Schnapsen is a challenging subject for the application of AI due to its partial information during the game and its strategic complexity [6]. Hybrid Intelligence has the potential to improve AI decision making in games like Schnapsen by leveraging both deterministic strategies and adaptive learning from game data.

2.3 Bots

In this research, comparisons with and references to existing bots will be made, so understanding their strategies is important.

Machine Learning bot (ML Bot): The Machine Learning bot is trained using machine learning techniques, specifically reinforcement learning or supervised learning, where it learns from a dataset of past games. After analyzing its previous games, it improves its ability to make decisions based on patterns and past outcomes. This bot excels in adapting to new situations, as it can learn and refine its strategies through experience. This is the bot that this experimental design aims to improve by adding heuristics to decide which move is best.

Rdeep Bot: This bot relies on deep search algorithms and strategic heuristics to evaluate moves. Its strength lies in deterministic situations where information is mostly complete, but it is computationally very expensive, especially in the first phase of the game.

Rand Bot: Finally, Rand Bot makes random decisions without any strategic reasoning. It serves as a baseline for testing other bots.

3 Research Question

The research question for this study is: *"To what extent does a Hybrid Intelligence (HI) framework that integrates domain-specific heuristics with machine learning improve decision quality and performance in imperfect-information sequential decision-making environments when evaluated against both stochastic and search-based opponents?"*

This research seeks to explore the impact of combining rule-based heuristics with machine learning techniques in a Schnapsen-playing AI agent. The hypothesis is that the addition of situation-specific decision rules, which are based on strategic elements of the game, will improve the performance of the Machine Learning Bot. The primary metric for evaluation will be the win rate of the Hybrid Intelligence Bot in comparison to the original Machine Learning Bot, where they will be separately compared to different bots. Tests will be performed comparing the Hybrid Intelligence and Machine Learning Bot to the Rdeep Bot and the Rand bot.

By addressing this question, the study will examine the potential benefits of Hybrid Intelligence in Schnapsen and other similar strategic games, where both deterministic strategies and adaptive decision-making are crucial. The goal is to determine whether the collaboration of machine learning and rule-based AI can result in a more effective and reliable game agent, capable of outplaying its purely data-driven counterpart. This work aims to contribute to the broader field

of AI research, particularly in the development of intelligent agents for games that involve incomplete information and complex decision-making.

4 Experimental Setup

4.1 Implementation & Training Details

The Machine Learning bot was trained using supervised learning on replay memory generated from simulated Schnapsen games. Training data was produced by running 100,000 self-play games between two RdeepBot bots with different strengths (`num_samples=20`, `depth=10` versus `num_samples=10`, `depth=5`), generating state-action-outcome tuples stored in `data/replay_memory.txt`.

The model architecture consists of a neural network (NN) with two hidden layers of sizes 100 and 50 neurons respectively, using the hyperbolic tangent activation function. Training was performed using scikit-learn’s `MLPClassifier` with a modified learning rate 0.001, L2 regularization strength 0.0001, early stopping enabled after 6 iterations without improvement, and a maximum of 200 training epochs. The trained model is serialized to `models/ml_bot_model.pkl` and loaded at runtime by the `MLPlayingBot` class.

The Hybrid Intelligence bot is implemented in `heuristic_ml_bot.py`, which extends the original ML bot by overriding the move selection logic to apply heuristics before falling back to machine learning predictions. Each heuristic is evaluated sequentially based on the current game phase, score difference, available actions, and trump status.

4.2 Training Data Generation and Evaluation Protocol

It is important to note that the Machine Learning bot was trained exclusively using replay data generated from self-play between two instances of the RdeepBot. During the training, the ML bot did not interact with either the RandBot or the Hybrid Intelligence bot, and no heuristic-based decisions were included in the training data. The training process consists solely of loading the pre-generated replay memory files and fitting the model parameters accordingly.

Post-training evaluation was conducted in separate simulation runs, where the trained ML bot and the Hybrid Intelligence bot were tested against the RandBot and the RdeepBot. These evaluation games were not written back into the replay memory and therefore did not influence the training process. This strict separation ensures that all reported performance results reflect generalization to unseen opponents and are not biased by data leakage from the evaluation phase.

Runtime performance was not used as an evaluation metric; only win rate was considered, as computational budgets differ significantly between bots, particularly between the search-based RdeepBot and the learning-based bots.

4.3 Selected Heuristics

For this experimental setup, a subset of rule-based heuristics was selected to try to improve the Machine Learning Bot for the game of Schnapsen. The heuristics were derived from strategies described by Martin Tompa in Winning Schnapsen [5], which outlines effective strategies to win a game of Schnapsen. Rather than implementing all recommended strategies simultaneously, multiple heuristic combinations were evaluated. Preliminary experiments indicated that different combinations of heuristics significantly affected performance.

The selected heuristics focus on score-aware and phase-aware decision making. The following strategies showed positive results during the preliminary experiments:

- **Heuristic 1 - Aggressive play when significantly ahead in Phase One:** when the bot is in Phase One and is leading by at least 20 points, it prioritizes playing high-impact actions by first attempting to play a marriage and otherwise playing a high-value card (rank 10 or higher) to close the talon as quickly as possible.
- **Heuristic 2 - Win a trick by following:** when the bot is not leading a trick and can win the trick by following suit, it plays the lowest-ranking card of that suit that still guarantees winning the trick.
- **Heuristic 3 - Early trump exchange:** when the bot is leading during Phase One and a trump exchange is available, it prioritizes the trump exchange.
- **Heuristic 4 - Aggressive play when significantly behind:** when the opponent is leading by at least 20 points, the bot prioritizes high-impact actions by first attempting to play a marriage and otherwise playing a high-value trump card to reduce the score gap.
- **Heuristic 5 - Conservative play when significantly ahead:** when the bot is leading by at least 20 points, it avoids unnecessary risks by preserving marriages and trump cards, and instead plays lower-value non-trump cards.

While optimizing the Hybrid Intelligence bot, a counter has been implemented in the code that measured how often each heuristic was used and how often it fell back on Machine Learning. This frequency was measured in a tournament against the Machine Learning bot that consisted of 1,000,000 games of Schnapsen. The results are displayed in the figure below.

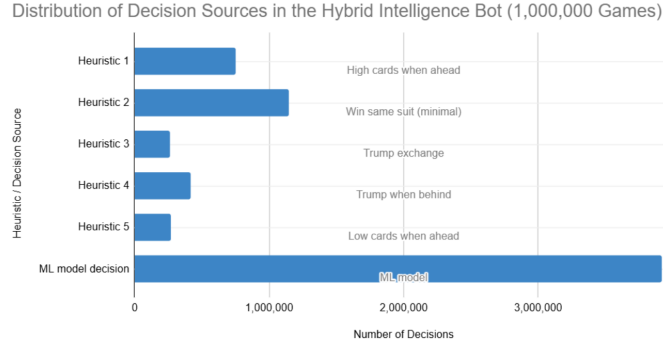


Fig. 1. Graph showing the use frequency of each heuristic

4.4 Experimental Tournaments

To evaluate the performance of the Hybrid Intelligence Bot, a series of Schnapsen tournaments were conducted comparing its win rates to those of the Machine Learning Bot and established baseline opponents. All experiments were performed on a 2025 MacBook Pro with an M5 Chip and 24 GB of memory, using Python 3.14.2 and the Schnapsen game engine.

The experimental setup consists of three types of comparisons. First, tournaments were conducted between the Machine Learning bot and the Hybrid Intelligence Bot. This is done to gain insight into whether the Hybrid Intelligence Bot can outperform its own machine learning component. Second, both the Machine Learning bot and the Hybrid Intelligence Bot were independently evaluated against the Rdeep Bot. This is to analyze whether win rates against a strong opponent increase after the implementation of rule-based strategies. The num_samples of RDeep Bot was set to 10 with a search depth of 5, which corresponds to approximately 10 Monte Carlo rollouts per decision and a fixed playout depth of 5 moves. Lastly, both bots were also evaluated against the Rand Bot, which provides a baseline representing win rates against a bot that uses non-strategic gameplay.

By simulating real gameplay conditions, these tournaments allow for a controlled comparison of bot performance across varying levels of opponent strength. The primary evaluation metric is the win rate, which provides a clear and objective measure of performance. A higher win rate directly reflects more effective strategic behavior, making it suitable to assess the impact of Hybrid Intelligence.

4.5 Tournament Results

Machine Learning bot vs. Hybrid Intelligence Bot A large-scale head-to-head tournament was conducted between the Machine Learning bot and the Hybrid Intelligence Bot, consisting of a total of 1,000,000 games. In this direct

comparison, the Hybrid Intelligence Bot achieved 555,990 wins, which corresponds to a win rate of 55.60%, while the Machine Learning bot achieved 444,010 wins, which corresponds to a win rate of 44.40%. This represents an absolute improvement of 11.20 percentage points in favor of the Hybrid Intelligence Bot. in relative terms, the Hybrid Intelligence Bot outperformed the Machine Learning bot by approximately 25.2%, calculated as

$$\frac{55.60 - 44.40}{44.40} \approx 0.252 \quad (1)$$

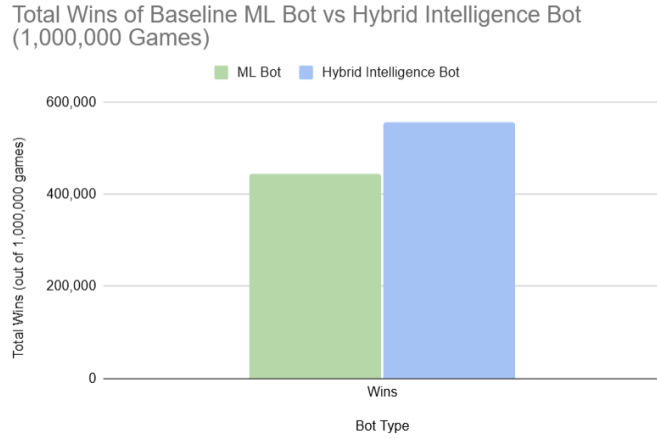


Fig. 2. Win-Rates of Machine Learning and Hybrid Intelligence Bot against each other

Performance Against Rdeep Bot To evaluate performance against a stronger search-based opponent, both the Machine Learning bot and the Hybrid Intelligence Bot were independently evaluated against the Rdeep Bot in 10 sets of tournaments. Each set consisted of 2,000 games for both bots. Across all tournament sets, the Hybrid Intelligence Bot achieved a higher win rate than the Machine Learning bot. The Machine Learning bot achieved an average win rate of 17.06% against the Rdeep Bot, while the Hybrid Intelligence Bot achieved an average win rate of 19.63%. This corresponds to an average relative improvement of approximately $(19.63 - 17.06) / 17.06 = 15.06\%$ in the win rate against the Rdeep Bot. This difference was later confirmed to be not only statistically significant but also practically large, with an effect size of Cohen’s $d \approx 2.83$. The consistent improvement across all tournament sets, as shown in the graph below, indicates that the heuristics provide meaningful strategic advantages even against a strong opponent that relies on Monte Carlo rollouts and heuristic evaluations.

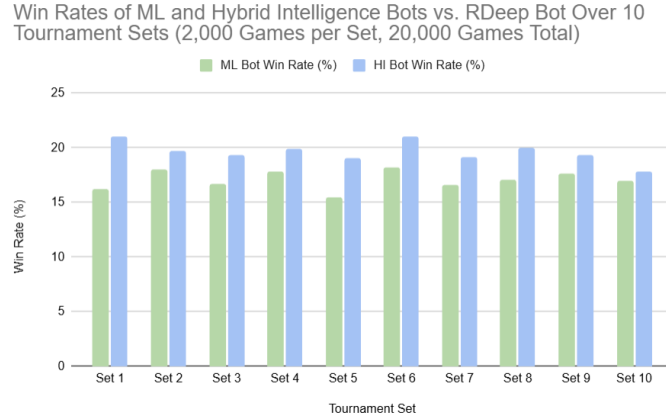


Fig. 3. Graph showing the win rates of ML and HI Bots against Rdeep Bot

Performance against Rand Bot To assess performance against a stochastic adversary, the Machine Learning bot and the Hybrid Intelligence Bot were evaluated separately against the RandBot across 10 tournament sets. Each set of tournaments consisted of 2,000 games per bot.

The results show that the Hybrid Intelligence Bot outperformed the Machine Learning bot in every set of tournaments. On average, the Machine Learning bot obtained a win rate of 59.24% against the RandBot, while the Hybrid Intelligence Bot achieved an average win rate of 65.87%. This represents a relative increase in the win rate of approximately $(65.87 - 59.24)/59.24 = 11.19\%$.

The consistent improvement across all tournament sets, as seen in the graph figure below, indicates that the integration of domain-specific heuristics enhances the robustness and decision quality of the learned policy, even when facing a stochastic opponent whose actions are not guided by strategic search or long-term planning.

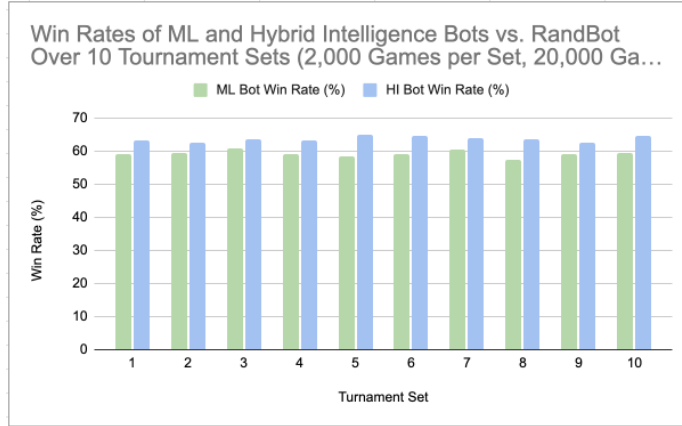


Fig. 4. Graph showing the win rates of ML and HI Bots against Rand Bot

4.6 Statistical Tests

Motivation For Evaluation Of The HI Bots' Performance Using Rdeep

Bot For statistical tests, a comparison is made between the Machine Learning bot and the Hybrid Intelligence bot against the RDeep Bot. This is because the Rdeep Bot strategy is more useful as an effective benchmark to evaluate improvements in the Hybrid Intelligence bot, which is due to the following.

The Rdeep Bot uses Monte Carlo rollouts to simulate multiple possible outcomes for each move. This makes it robust and capable of handling a wide variety of game scenarios, ensuring that the ML and HI Bots are tested against diverse and unpredictable gameplay.[8] Additionally, the Rdeep Bot incorporates a heuristic evaluation function to prioritize moves that maximize player scoring advantage; this heuristic introduces a level of strategic decision-making, making Rdeep Bot more challenging than purely random or simplistic bots. This is a valuable benchmark feature, since comparing the two bots against Rdeep tests their ability to outperform a bot that uses well-defined heuristics, providing a measure of strategic depth. Furthermore, the Rdeep Bot design is based on randomized rollouts seeded with a consistent random generator that ensures that Rdeep Bot's gameplay is reproducible for fair comparisons while still being non-deterministic enough to simulate realistic gameplay variability. Rdeep's design principles randomized rollouts and heuristic evaluations are effective in a range of game states and scenarios. This makes Rdeep a balanced benchmark, not biased towards specific strategies, allowing for fair evaluation of the bots' general performance.

The tournament results between the ML and HI bots and the Rand Bot were neither subject to the same statistical tests, since the Rand Bot is used solely as a rough baseline to check the performance of the heuristic Machine Learning bot against the original Machine Learning bot. To evaluate the statistical significance of the difference between the win rates of the bots against the Rdeep Bot,

the unpaired t-test was used. The purpose of using the t-test on the set of bot win rate data is to compare the performance of the two bots across the multiple sets of tournaments (in this case it is 10 sets of 2000 games played between the Machine Learning bot and Rdeep Bot, and the Hybrid Intelligence bot and the Rdeep Bot). The unpaired t-test was used (instead of a paired t-test) because the Machine Learning bot and the Hybrid Intelligence bot are two different groups. This is because the performance of one bot is independent of the other, since their win rates are calculated separately, based on games played independently against the Rdeep Bot. This t-test is two-tailed because it is testing for a difference in win rates between the two aforementioned bots, and it does not assume beforehand which bot will perform better. A specific directional hypothesis is not made for the purpose of this test, as there is no assumption that the Hybrid Intelligence bot would always have a higher win rate over the Machine Learning bot, in order to make this an unbiased test and prevent skewed results.

Hypotheses: Null Hypothesis (H_0): The mean win rate of the ML Bot (μ_1) is equal to the mean win rate of the HI Bot (μ_2).

$$H_0 : \mu_1 = \mu_2$$

Alternative Hypothesis (H_a): The mean win rate of the ML Bot (μ_1) is not equal to the mean win rate of the HI Bot (μ_2).

$$H_a : \mu_1 \neq \mu_2$$

Calculation: To carry out this statistical test, a dataset is created in the form of two lists of win rates, one for the Machine Learning bot against the Rdeep Bot and one for the win rates of the Hybrid Intelligence bot against the Rdeep Bot (both lists are numerical win percentages of the ML and HI bots over the Rdeep Bot). The unpaired two-tailed t-test is performed on this data set. Given the approximately symmetric distribution of win rates across tournament sets and the equal sample sizes for both bots, the normality assumption required by the t-test is considered appropriate.

ML Bot Win Rates (%): [16.25, 18.00, 16.65, 17.80, 15.45, 18.20, 16.55, 17.05, 17.65, 17.00]

HI Bot Win Rates (%): [21.05, 19.70, 19.35, 19.90, 19.05, 21.00, 19.10, 20.00, 19.30, 17.85]

The degrees of freedom for this test are: $df = x_1 + x_2 - 2$, thus in a sample of 10 sets of average win rate of the ML and HI bots against the Rand Bot of 2000 games, $df = 10 + 10 - 2 = 18$. Thus, the degree of freedom is 18.

The test statistic t equals -6.3290, which is not in the 95% region of acceptance: [-2.1009, 2.1009]. $x_1 - x_2 = -2.57$, is not in the 95% region of acceptance: [-1.3197, 1.3197].

The p-value equals 5.785×10^{-6} , which means that the chance of a type I error (rejecting a correct null hypothesis, H_0) is very small: 0.000005785. The

smaller the p-value, the more it supports the alternative hypothesis, H_a . For a two-tailed test, the p-value is set as $p < 0.05$. The extremely high absolute value of t indicates that the p-value is close to zero ($p < 0.001$). This leads to the conclusion that since the p-value is much less than the standard significance level ($\alpha = 0.05$), the null hypothesis is rejected (H_0) and there is strong statistical evidence to conclude that the ML and HI bots win rates against the Rdeep Bot are significantly different. To quantify the practical significance of this result, Cohen’s d was calculated as an estimate of the effect size. Using the difference in mean win rates and the pooled standard deviation between set-ups of tournaments, $d \approx 2.83$ was obtained, indicating a very large effect.

In conclusion, since the p-value $< \alpha$, H_0 is rejected, the average of the Machine Learning bot win rate against Rdeep Bot is considered not equal to the average of the Hybrid Intelligence bot win rate against Rdeep Bot to a statistically significant degree. This suggests strong evidence that the implementation of Hybrid Intelligence in game play, by introducing heuristics to a Machine Learning bot model, is successful in improving the bot’s performance.

Following these findings, the magnitude of the observed difference between the two agents indicates that the improvement is not only statistically significant but also practically meaningful. While both agents rely on machine learning to guide their decision-making process, the Hybrid Intelligence Bot benefits from the integration of domain-specific, phase-aware heuristics that complement the learned policy in strategically critical situations. This hybrid approach enables the bot to exploit structured knowledge about the game dynamics, such as card strength, trump management, and score-dependent risk behavior, which may not be fully captured by the data-driven model alone.

These results support the hypothesis that combining learned policies with carefully designed heuristic rules can lead to more robust and consistent performance in partially observable environments. In particular, the improvement against a strong search-based opponent such as RdeepBot demonstrates that heuristic guidance can enhance long-term planning and tactical stability, even when facing adversaries that employ Monte Carlo rollouts and heuristic evaluation functions themselves.

Overall, this experiment provides empirical evidence that hybrid intelligence constitutes an effective design paradigm for competitive game-playing agents in complex stochastic domains. By leveraging both statistical learning and expert knowledge, such systems can achieve superior performance compared to purely machine-learning-based approaches, especially in scenarios where imperfect information and strategic uncertainty play a central role.

5 Conclusion

The findings from our experimental tournaments show the effectiveness of applying Hybrid Intelligence in bot design. An unpaired two-tailed t-test revealed a statistically significant improvement in the win rate of the Hybrid Intelligence bot over the original Machine Learning bot against Rdeep Bot (p-value < 0.05).

The relative improvement rate expressed as a percentage shows that the addition of heuristics to the Machine Learning bot relatively increased its win rate against Machine Learning bot by 25.2%, against Rdeep Bot it increased by 15.06% and against Rand Bot there was a relative increase of 11.19%.

By integrating rule-based strategies with machine learning, the Hybrid Intelligence Bot consistently outperformed the original Machine Learning Bot, particularly against Rdeep Bot. Although Rdeep Bot remained a stronger competitor, the Hybrid Intelligence bot showed clear improvements over the Machine Learning bot, proving its potential to bridge the gap between purely statistical approaches and human-like strategies. These results confirm the value of Hybrid Intelligence in improving the performance of decision-making systems in card games like Schnapsen, where information is incomplete. This serves as a valuable implication for the field of developing effective game agents in the field of artificial intelligence and suggests that hybrid intelligence is a valuable research path for opportunities in advancement of decision making computational agents.

6 Limitations and Future Work

Although the results demonstrate the potential of the Hybrid Intelligence Bot, several limitations provide opportunities for future work. Firstly, the experiments used only one set of heuristics, which may not represent the optimal configuration. Exploring alternative heuristics or combinations of heuristics could lead to better performance and uncover additional improvements. Thus, a follow-up research question that would contribute to this paper would be: *"What heuristics optimize a Hybrid Intelligence bots' performance in playing Schnapsen?"*. Additionally, despite the Hybrid Intelligence Bot outperforming the original Machine Learning Bot, it was still outperformed by Rdeep Bot. Future research could focus on refining the Hybrid Intelligence approach by integrating more advanced heuristics or improving the machine learning algorithm(s). Incorporating opponent modeling may also enhance competition against stronger adversaries like Rdeep Bot, as this has been proven to work in other turn-based games. [9]

References

1. Aksjonov, A., Kyrki, V.: A safety-critical decision making and control framework combining machine learning and rule-based algorithms. arXiv preprint arXiv:2201.12819 (2022) <https://doi.org/10.48550/arXiv.2201.12819>
2. Salzer, T.: Reinforcement learning for games with imperfect information. MSc thesis, Technische Universität Wien (2023). <http://www.heitzinger.info/Papers/Tobias%20Salzer%20-%20MSc%20thesis.pdf>
3. Sugiyanto, Fernando, G., Tai, W.-K.: A rule-based AI method for an agent playing big two. *Applied Sciences* **11**(9), 4206 (2021). <https://doi.org/10.3390/app11094206>
4. Singh, C.: Machine learning in pattern recognition. *European Journal of Engineering and Technology Research* **8**(2), 63–68 (2023). <https://doi.org/10.24018/ejeng.2023.8.2.3025>
5. Tompa, M.: Winning Schnapsen. Psellos (2015). <https://psellos.com/schnapsen/blog/2015/09/book.html>
6. Kovařík, V., Schmid, M., Burch, N., Bowling, M., Lisý, V.: Rethinking formal models of partially observable multiagent decision making. *Artificial Intelligence* **303**, 103645 (2022). <https://doi.org/10.1016/j.artint.2021.103645>
7. Truong, T.: Monte Carlo Tree Search–Minimax Hybrid in AlphaZero. Bachelor’s Thesis, Technical University of Darmstadt (2023). <https://ml-research.github.io/papers/truong2023monte.pdf>
8. Dunn, W. L., Shultis, J. K.: *Exploring Monte Carlo Methods* 2nd ed. Elsevier Science, Amsterdam (2022). ISBN 978-0-12-819739-4. <https://doi.org/10.1016/C2017-0-01717-2>
9. Yu, X., Jiang, J., Zhang, W., Jiang, H., Lu, Z.: Model-based opponent modeling. In: *Advances in Neural Information Processing Systems* **35**, pp. 1–13 (2022). https://proceedings.neurips.cc/paper_files/paper/2022/file/b528459c99e929718a7d7e1697253d7f-Paper-Conference.pdf
10. Yuan, W., et al.: Ensemble strategy learning for imperfect information games. *Neurocomputing* **541**, 126514 (2023). <https://doi.org/10.1016/j.neucom.2023.126514> <https://www.sciencedirect.com/science/article/pii/S0925231223003648>
11. Schmid, M., et al.: Student of games: A unified learning algorithm for both perfect and imperfect information games. *Science Advances* **9**(46), eadg3256 (2023). <https://doi.org/10.1126/sciadv.adg3256>

7 Appendix

A Machine Learning Model Training Details

```

from schnapsen.bots.ml_bot import train_ML_model
import pathlib

def main():
    replay_memory_path = pathlib.Path("data/replay_memory.txt")
    model_save_path = pathlib.Path("models/ml_bot_model.pkl")
    model_save_path.parent.mkdir(exist_ok=True, parents=True)

    # training the ML model using replay memory
    print("Starting training...")
    train_ML_model(
        replay_memory_location=replay_memory_path,
        model_location=model_save_path,
        model_class="NN" # we chose to go with the NN model for our experiments
        # (apparently it performs slightly better than the LR model)
    )
    print(f"Model saved at {model_save_path}")

if __name__ == "__main__":
    main()

```

B Hybrid Intelligence Heuristic Rules

```

# ===== HEURISTIC START =====
    opponent_score = perspective.get_opponent_score()
    opponent_points = opponent_score.direct_points + opponent_score.pending_points

    # get our total points
    my_score = perspective.get_my_score()
    my_points = my_score.direct_points + my_score.pending_points

    # ===== HEURISTIC 1: Aggressive Play When Ahead =====
    # If I'm ahead by 20+ points in Phase 1, play high-value cards
    # Strategy = End the game quickly by depleting the talon
    if perspective.get_phase() == GamePhase.ONE and
    (my_points - opponent_points >= 20):
        # First priority: Play marriages (40 or 20 points)
        for move in perspective.valid_moves():
            if move.is_marriage():
                self.heuristic_counters["heuristic_1"] += 1
                return move

```

```

        # Second priority: Play high-value cards (Ace=11, Ten=10)
        if hasattr(move, 'card') and move.card.rank.value >= 10:
            self.heuristic_counters["heuristic_1"] += 1
            return move

# ===== HEURISTIC 2: Minimal Winning Card =====
# When following and can win, play the lowest card that still wins
# Strategy = Conserve high cards for later while still winning the trick
if (leader_move is not None) and (not perspective.am_i_leader()) and
hasattr(leader_move, "card"):
    leader_card = leader_move.card

    # Find all cards of same suit that beat the leader's card
    winning_same_suit = [
        m for m in perspective.valid_moves()
        if hasattr(m, "card")
        and m.card.suit == leader_card.suit # same suit
        and m.card.rank.value > leader_card.rank.value # higher rank
    ]

    # if we can win with same suit, play the lowest winning card
    if winning_same_suit:
        self.heuristic_counters["heuristic_2"] += 1
        return min(winning_same_suit, key=lambda m: m.card.rank.value)

# ===== HEURISTIC 3: Trump Exchange Opportunity =====
# In Phase 1 as leader, exchange low trump (Jack) for higher trump card
# Strategy: Improve trump strength at minimal cost
if perspective.am_i_leader() and perspective.get_phase() == GamePhase.ONE:
    for move in perspective.valid_moves():
        if move.is_trump_exchange():
            self.heuristic_counters["heuristic_3"] += 1
            return move

# ===== HEURISTIC 4: Catch-Up Strategy When Behind =====
# If opponent has 20+ more points, play high-value moves to catch up
# Strategy = Prioritize marriages and high trumps
if opponent_points - my_points >= 20:
    for move in perspective.valid_moves():
        # Marriages give 20 or 40 points instantly
        if move.is_marriage():
            self.heuristic_counters["heuristic_4"] += 1
            return move
    # High trumps (Queen, King, Ace) can win tricks and score points

```

```

        if hasattr(move, 'card') and
        move.card.suit == perspective.get_trump_suit()
        and move.card.rank.value > 10:
            self.heuristic_counters["heuristic_4"] += 1
            return move

# ===== HEEURISTIC 5: Preserve Advantage When Ahead =====
# If we have 20+ point lead, play low-value non-trump cards
# Strategy = Save marriages and trumps for later, play mediocre cards
if my_points - opponent_points >= 20:
    for move in perspective.valid_moves():
        # Play non-marriage, low-value, non-trump cards
        if (not move.is_marriage()) and (move.card.rank.value <= 10) and
        (move.card.suit != perspective.get_trump_suit()):
            self.heuristic_counters["heuristic_5"] += 1
            return move # Preserve marriages & trumps, play mediocre cards

```

C Experimental Outcomes Tables

Tables 1 - 5 show complete tournament results.

Table 1. Hybrid vs. ML Bot Win Rates (1M games).

Bot Type	Wins	Total Games
ML Bot	444,010	1,000,000
Heuristic ML Bot (Hybrid)	555,990	1,000,000

Table 2. Heuristic Strategy Usage (1M games).

Decision Source	Description	Count	Percentage (%)
Heuristic 1	High cards when ahead	751321	11.11
Heuristic 2	Win same suit (minimal)	1146651	16.95
Heuristic 3	Trump exchange	259259	3.83
Heuristic 4	Trump when behind	419211	6.20
Heuristic 5	Low cards when ahead	271430	4.01
ML decision	ML model decision (fallback)	3915350	57.89
Total heuristic decisions		2847872	42.11
Total ML decisions		3915350	57.89
Total decisions		6763222	100.00

Table 3. Overall Win Rates vs. Rand Bot (10×2000 games).

Bot	Total Wins	Total Games	Win (%)
ML Bot	11847	20000	59.24
HI Bot	13173	20000	65.87

Table 4. Overall Win Rates vs. Rand Bot (10×2000 games).

Bot	Avg. Win Rate (%)
ML Bot	59.24
HI Bot	65.87

Table 5. Overall Win Rates vs. Rdeep Bot (10×2000 games).

Bot	Avg. Win Rate (%)
ML Bot	17.06
HI Bot	19.63